

Document Number: **P0670R2**, ISO/IEC JTC1 SC22 WG21
Audience: EWG, LEWG
Date: 2017-11-08
Authors: Matúš Chochlík (chochlik@gmail.com)
Axel Naumann (axel@cern.ch)
David Sankel (camior@gmail.com)

Static reflection of functions

Table of Contents

- [Introduction](#)
- [Interplay with other proposals](#)
- [Concepts and Operations](#)
- [Acknowledgments](#)
- [References](#)

Introduction

[P0194](#) introduced static reflection for types and variables. This paper adds static reflection of functions.

Reflection proposed here behaves as follows:

The functionality introduced here allows for reflection of calls of concrete functions. This enables, for instance, GUI generation, building a catalogue for remote procedure calls, documentation generation, and signal/slot frameworks. Like P0194, this proposal omits attributes, templates, and reification (i.e. conversion of a meta object to the base level): all warrant a separate paper. We would welcome a paper especially on static reflection of attributes, matching the interface-style of P0194 and this paper! Linkage and friends will be part of a follow-up paper to P0194; they will have a combined "effect" on P0194 and this paper.

Interplay with other proposals

Most notably, this proposal relies on P0194 and the Concepts TS.

[P0385](#) discusses use cases, rationale, design decisions, and the future evolution of the proposed reflection facility. It also has usage examples and replies to frequently asked questions.

Concepts and Operations

Following P0194's lead, function reflection requires a couple new concepts to restrict operations on the meta level. It builds upon the concepts introduced by P0194 and, like P0194, all declarations are inside the `std::reflect` namespace. Some of the concepts below extend P0194's concepts. Whenever a concept A requires another concept B, the operations defined for concept B are also available for concept A, building a graph similar to an inheritance graph.

Proper wording will be provided when needed (and when P0194 has progressed through LWG, increasing to the authors' experience on wording of reflection papers); we believe that missing wording should not hinder SG7's design discussion. The wording is planned to be similar to that of P0194; specifically, it uses nested `using` and `using namespace` entities, with the usual `using` declarations and `using namespace`.

Extending operands of

P0194 allows certain types and variables as operands of `std::reflect::operator()`. This paper extends this set:

- invoking `std::reflect::operator()` on a function call expression generates a `std::reflect::function` object;
- invoking `std::reflect::operator()` on a closure type generates a `std::reflect::function` object;
- invoking `std::reflect::operator()` on a parameter name generates a `std::reflect::parameter` object;
- invoking `std::reflect::operator()` on a lambda capture generates a `std::reflect::capture` object;

The way to obtain a `std::reflect::function` object might seem complicated. But instead of inventing a new syntax, for instance `std::reflect::operator()(<function call>)`, re-using the mechanism of matching the address of an overloaded function name `<function name>`, or syntax similar to function definition `<function name>(<parameters>)`, we rely on the existing, familiar rules for function overload resolution and generate the `std::reflect::function` object only when a concrete overload is selected. This approach also works for constructors and operators, for instance `std::reflect::operator()(<type>)` reflects the call of the default constructor of `<type>`; `std::reflect::operator()(<type>, <operator>)` reflects the call of the built-in addition operator for type `<type>`.

Requires `std::reflect::function` and `std::reflect::parameter`. Represents a function parameter. Unlike `std::reflect::parameter`, it does not offer a `std::reflect::parameter::operator()` interface. The name of a parameter is one of the names used in an unspecified declaration. If at least one of the declarations does not specify the parameter name, then `std::reflect::parameter::name` is allowed to return an empty string. Its scope is the `std::reflect::function` declaring this.

Given the lack of relevance that C++ attributes to parameter names in function declarations one might ask: "o really?!" We believe that the answer is "yes": parameter names often carry significant meaning. Examples:

The bare combination of type and index is almost meaningless in these cases. There are many reflection applications that can benefit from this, for instance:

- remote procedure calls identifying parameters by name;
- code exposing C++ functions to scripting frameworks;
- GUI generation, converting parameter names to text box labels;
- documentation generation, referencing function parameters in a meaningful way.

To put it differently: functions without parameter names are like classes without member names. And `std::function` is not a replacement for classes.

Another common objection is that multiple declarations could potentially have different parameter names ascribed. This concern is mitigated in two ways:

1. Modern coding conventions have the declarations for a particular function showing up in exactly one header file.
2. Modern coding conventions discourage the use of different argument names between function declarations (in a header) and function definitions (in a '.cpp' file). Dedicated compiler warnings exist to protect against this case.

Operations

Indicates that a `std::function` reflects an ellipsis.

Indicates that a `std::function` has a default value.

Requires `std::function`, `std::weak_ptr` and `std::shared_ptr`. Represents a function or lambda, including operators, constructors and destructors - i.e. anything this paper is dealing with.

Operations

Returns an `std::function` of `std::weak_ptr` s of the reflected `std::function`.

Returns whether the function was declared as `constexpr` or `consteval`, respectively.

Returns whether the function is an inline function. With `constexpr`, `consteval` is `true` for both `constexpr` and `consteval`.

Returns whether the function was defined as `constexpr` before the invocation of `constexpr`.

Requires `constexpr`. Reflects a call of a concrete function or other callable.

Operations

Returns the `constexpr` that was invoked in a `constexpr`.

Requires `constexpr` and `constexpr`. Represents a function or lambda, excluding constructors and destructors.

Operations

Returns a pointer to the function. This is a pointer-to-member for non-static member functions (including lambda calls), and a function pointer otherwise. It is ill-formed to invoke this for deleted functions. Example:

`&f` holds the address of `f`.

Requires `std::atomic` and `std::memory_order`. Represents a member function, excluding constructors and destructors.

Operations

Returns whether this is a static member function.

Returns whether the function is declared as `constexpr` or `consteval`, respectively.

Returns whether the function is declared as with a *ref-qualifier* being `&` or `&&`, respectively.

Returns whether the function is a virtual or pure virtual function, respectively. For

the value of `is_virtual` is `true` if the function is virtual, `false` otherwise, irrespectively of whether `is_pure_virtual` is implicit or explicit.

Returns whether the function is declared as `constexpr` or `consteval`, respectively.

Requires `constexpr`. Represents a special member function.

Operations

Returns whether the special member function is known to be implicitly declared at the point of invocation of `constexpr`.

Returns whether the function is defined as `constexpr` before the invocation of `constexpr`, independently of whether the special member function is implicitly or explicitly declared.

Requires `constexpr` and `constexpr`. Represents a constructor. The base name of the constructor is the base name of the constructor's class. Even though the standard explicitly says that constructors do not have a name, for usability purposes (e.g. generating messages), having them state the class name is a usability improvement.

Some instances of `constexpr` might also satisfy `constexpr`.

Operations

Returns whether the constructor is known to be declared as `constexpr`.

Requires `constexpr`, `constexpr` and `constexpr`. Represents a destructor. The base name is the base name of the destructor's class, prefixed with `~`.

Returns whether the destructor is a virtual or pure virtual function, respectively. For

the value of `is_virtual_destructor` is `true` or `false`,
irrespectively of whether `is_virtual_destructor` is implicit or explicit.

Requires `is_virtual_destructor`. Some instances might implement `is_virtual_destructor` or
instance `is_virtual_destructor`. Represents an operator. The base name is the operator "symbol", for
instance `is_virtual_destructor`.

Requires `is_virtual_destructor`. Represents a conversion operator. The base name is the base name of the
operator's target type, for instance `is_virtual_destructor`.

Operations

Returns whether the function is declared as `is_virtual_destructor`.

Requires `is_virtual_destructor`. Represents a closure type, excluding those for generic lambdas. Its base name
is the empty string.

Operations

Returns an `bool` of `bool` s.

Returns whether the *capture-default* is `bool` .

Returns whether the *capture-default* is `bool` .

Returns `bool` if the lambda was declared as `bool` , `bool` .

Requires `bool` . Represents a lambda capture as introduced by the capture list or by capture defaults. The `bool` 's scope is its `bool` .

Operations

Returns whether the entity was captured explicitly.

Returns whether the entity is an *init-capture*.

Extending

This proposal adds the following interfaces to the `bool` concept of P0194:

Returns an `std::vector` of `std::string`s representing a class's public member functions (for `std::string` invocation of `std::string` (for `std::string`) and all member functions, irrespective of their accessibility (for `std::string`). This includes *static* member functions, but not *friend* functions.

Returns an `std::vector` of a class's `std::string`s, `std::string`s and `std::string`s, respectively.

Acknowledgments

Thanks to Jackie Kay who provided valuable feedback, criticism and suggestions!

References

1. *Static reflection. Rationale, design and evolution.* [p0385](#)
2. *Static reflection in a nutshell.* [p0578](#)